

Tai Deepshika . S

192511286

CO2 - AT1

1) Addition and Subtraction using 2's Complement

Given:

$$A = +45$$

$$B = -23$$

sol:

$$\begin{array}{r}
 45 \qquad \qquad 00101101_2 \\
 +23 \qquad \qquad 00010111 \\
 -23 \qquad \qquad 11101000
 \end{array}$$

Add:

$$\begin{array}{r}
 \begin{array}{ccccccc}
 & 1 & 1 & & 1 & & 1 \\
 & 0 & 0 & 1 & 0 & 1 & 1 \\
 & 1 & 1 & 1 & 0 & 1 & 0 \\
 \hline
 1 & 0 & 0 & 0 & 1 & 0 & 1
 \end{array} \\
 \begin{array}{l}
 2^0 \times 1 = 1 \\
 1 \times 2^1 = 2 \\
 1 \times 2^2 = 4 \\
 0 \times 2^3 = 0 \\
 1 \times 2^4 = 16 \\
 \hline
 22
 \end{array}
 \end{array}$$

Addition:
 45 0 0 1 0 1 1 0 1
 23 0 0 0 1 0 1 1 1

 0 1 0 0 0 1 0 0

Ans: 68

Overflow Detection:

- * No overflow in both operations because overflow occurs,
- * Two positive numbers produce a negative result, or
- * Two negative numbers produce a positive result.

4-bit Carry Look-Ahead Adder (CLA)

Given:

Two 4-bit binary numbers:

$$A = 1010$$

$$B = 0110$$

Soln:

Generate: 0100

Propagate = 0011

Sum : 0 0 0 0

Result : 10000_2 (16_{10})

Comparison :

Ripple Carry Adder	Carry Look Ahead Adder
Carry propagates one stage at a time	Carries are computed simultaneously
Higher delay	Very low delay
Simple design	More Complex hardware
Slower for large adders	Faster and efficient

CLA is faster:

- * Uses Generate (G) and Propagate (P) signals.
- * Calculates all carry bits in advance instead of waiting for previous carry.
- * Reduces computation time and improves processor

perform
Conclusion:

A CLA computes carries in parallel. Generate and Propagate signals, making it much faster than a Ripple Carry Adder.

3) Booth's Algorithm for Signed Multiplication
Given:

$$\ast \text{Multiplicand (M)} = -6$$

$$\ast \text{Multiplier (Q)} = +5$$

Soln:

$$-6 = 1010$$

$$6 = 0110$$

$$5 = 0101$$

Initial values:

$$\text{Accumulator} = 0000$$

$$\text{Multiplier} = 0101$$

$$\text{Booth bit} = 0$$

$$\text{No of bits} = 4$$

Booth's Rules

- $Q_0 Q_{-1} = 01 \rightarrow A = A + M$
- $Q_0 Q_{-1} = 10 \rightarrow A = A - M$
- $Q_0 Q_{-1} = 00 \text{ or } 11 \rightarrow \text{No operation}$

Combine A and Q :

$$AQ = 11100010_2$$

$$= 6 \times (+5) = -30$$

Role of Registers:

- Accumulator : Stores partial products
- Multiplier(Q) : Holds the multiplier bits
- Booth Bit : Helps determine whether to add, subtract.

Advantages:

- Efficiently handles signed numbers.
- Reduces the number of addition and subtraction operations.
- Widely used in modern processors for signed multiplication.

4) Division Using Restoring and Non-Restoring Division Algorithms.

Given:

$$\text{Dividend} = 13$$

$$13 = 1101_2$$

$$3 = 0011_2$$

Restoring Division Algorithm

$$\text{Remainder (A)} = 0000$$

$$\text{Dividend (Q)} = 1101$$

$$\text{Divisor (M)} = 0011$$

$$A \leftarrow A - M$$

* If $A \geq 0$, quotient bit = 1

* If $A < 0$, restore $A = A + M$ and

set quotient bit = 0

$$\text{Quotient} = 0100_2 (4)$$

$$\text{Remainder} = 0001_2 (1)$$

$$* A = 0000$$

$$* Q = 1101$$

$$* M = 0011$$

$$\text{Quotient} = 0100_2 \quad (4)$$

$$\text{Remainder} = 0001_2 \quad (1)$$

Comparison

Restoring Division	Non-Restoring Division
Restores remainder if negative.	No restoration after every step
More arithmetic operations	Fewer arithmetic operations
Slower execution	Faster execution
Higher hardware complexity	More efficient implementation

* Non Restoring Division is preferred because it eliminates repeated restoration operations.

* Both restoring & non restoring division is preferred because it produce same result.

5)

IEEE 754 floating - Point Representation

Given :

$$\text{Decimal number} = -13.25$$

Binary :

$$13 = 1101_2$$

$$0.25 = 0.01_2$$

Normalize

$$1101.01_2 = 1.10101 \times 2^3$$

A) Single Precision (32 bit)

* Sign bit = 1

* Exponent = $3 + 127 = 130$

$$10000010_2$$

B) Double Precision (64 bit)

* Sign bit = 1

* Exponent = $3 + 1023 = 1026$

$$= 10000000010_2$$

Floating Point Addition:

- * Compare the exponents
- * Shift the mantissa of the smaller exponent to align both exponents.
- * Add or subtract the mantissas.
- * Normalize the result
- * Store final sign & exponent

Issues:

Normalization: Keeps the number in standard form.

Rounding: Extra bits are rounded to fit.

Conclusion:

The decimal number -13.25 is represented in IEEE 754 format using Sign, Exponent & Mantissa.